



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°4

1. Desarrollar la clase `VentanaCombate` heredando de `JFrame`. Configurar sus propiedades principales: título, dimensiones de 800x600, acción por defecto al cerrar y posicionamiento centrado en la pantalla.
2. A partir de la clase `VentanaCombate` del ejercicio anterior, agregue un `JButton` con la etiqueta "Saludar".
 - a. Implemente un `ActionListener` mediante una clase anónima que, al presionar el botón, imprima por consola el mensaje "¡Hola, arena de combate!".
 - b. Agregue un segundo botón con la etiqueta "Reiniciar" e impleméntelo utilizando una expresión lambda, de manera que al presionarlo imprima por consola el mensaje "Combate reiniciado".
 - c. Agregue un tercer botón con la etiqueta "Salir" cuyo listener cierre la ventana al ser presionado.
3. Implemente una pequeña aplicación de un contador construyendo las siguientes tres clases, sin usar ninguna clase `Swing` salvo donde se indique explícitamente.
 - a. Cree una clase `Contador` con el atributo `valor` y los métodos `incrementar()`, `decrementar()` y `getValor()`.
 - b. Cree una clase `VentanaContador` que herede de `JFrame` y contenga un `JLabel` con el valor actual y dos `JButton`, uno con la etiqueta "+" y otro con "-".
 - c. Cree una clase `GestorContador` que reciba en su constructor las referencias a las dos clases anteriores, asigne los `ActionListener` correspondientes a los botones y actualice el `JLabel` cada vez que el valor cambie.
 - d. Agregue un tercer botón con la etiqueta "Reiniciar" que restablezca el valor a 0 y actualice la vista.
 - e. Indique en un comentario al inicio de cada una de las tres clases qué rol del patrón Modelo-Vista-Controlador cumple (Modelo, Vista o Controlador) y qué característica de su implementación lo evidencia.
4. Utilizando un `javax.swing.Timer`, desarrolle una pequeña animación.
 - a. Cree un `JPanel` personalizado que dibuje una figura (por ejemplo un círculo o un rectángulo) en una posición dada por la variable `posicionX`.
 - b. Configure un `Timer` con un intervalo fijo (por ejemplo, 16 milisegundos) que en cada tick incremente `posicionX` y luego refresque el panel.
 - c. Haga que la figura rebote al alcanzar los bordes de la ventana, invirtiendo su dirección de desplazamiento.
 - d. Agregue un `JButton` con la etiqueta "Pausar/Reanudar" que detenga o reinicie el `Timer` alternadamente cada vez que se lo presione.



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°4

- e. Una vez que la animación funcione, indique en un comentario que concepto acaba de implementar al combinar una actualización periódica del estado con el refresco de la pantalla.
5. Diseñar una interfaz gráfica de usuario (IGU) mediante JFrame para simular el desplazamiento horizontal y el salto de un personaje en un juego de plataformas.
 - a. Crear la clase Personaje, la cual debe incluir los atributos posicionX, posicionY, fuerzaSalto y la variable booleana estaEnElSuelo.
 - b. Utilizar un rectángulo de color para la representación visual del personaje en pantalla.
 - c. Configurar un KeyListener en la ventana principal con el fin de que:
 - i. Las flechas direccionales (izquierda y derecha) modifiquen posicionX para mover el rectángulo.
 - ii. La tecla Espacio ejecute el método saltar(), alterando posicionY únicamente si estaEnElSuelo es true.
6. Crear una interfaz gráfica de usuario (IGU) que represente el panel de control de un vehículo de competición.
 - a. Definir la clase Auto con las variables velocidadActual (entre 0 y 220 km/h), limiteVelocidad y nivelCombustible (de 0 a 100%).
 - b. Implementar un componente JProgressBar destinado a simular el Velocímetro con una escala de 0 a 220.
 - c. Incluir otro JProgressBar para visualizar el Combustible, parametrizado de 0 a 100%.
 - d. Agregar dos botones de tipo JButton, uno con la etiqueta "Acelerar" y el otro con "Frenar".
 - e. Configurar la acción del botón "Acelerar" para aumentar la velocidad y restar combustible en una relación proporcional.
 - f. Programar el botón "Frenar" de modo que reduzca la velocidad de manera paulatina.
7. Agregue un bucle de refresco en tiempo real para la IGU.
 - a. Defina un temporizador con un intervalo de 16 milisegundos.
 - b. Durante cada pulsación o tick del temporizador, ejecute los siguientes pasos:
 - i. Modifique la lógica interna del modelo.
 - ii. Invoque el método repaint() en la ventana o panel para obligar a la renovación gráfica.



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°4

8. Restructure la simulación de combate desarrollada en los TPs anteriores aplicando estrictamente el patrón de diseño MVC:
- **Modelo:** Utilice las clases Héroe y Enemigo (Estas clases no pueden implementar nada del paquete Swing).
 - **Vista:** Cree una clase VentanaJuego que hereda de JFrame. Debe contener lo siguiente:
 - Paneles para el Héroe y Enemigo con barras de vida.
 - Botones de acción para los ataques y curaciones.
 - **Controlador:** Cree la clase CombateController que servirá de puente entre el modelo y la vista.
 - Debe recibir en su constructor las referencias del modelo y de la vista.
 - Asigna a los listeners de evento (ActionListener) a los botones de la vista.